

CS253 Assignment 1

Memory-Efficient Versioned File Indexer

Piyush Ghayal
Roll Number: 240748

1 Introduction

Large log files are commonly encountered in modern computing systems. Processing such datasets efficiently requires careful memory management because loading entire files into memory may lead to excessive resource usage. This project implements a memory-efficient versioned file indexer in C++. The system processes large text files incrementally using a fixed-size buffer ranging from 256 KB to 1024 KB. The file is read chunk by chunk, ensuring that the entire file is never loaded into memory at once.

The system builds a word-level index that records how many times each word appears in the dataset. Words are defined as contiguous sequences of alphanumeric characters and are treated in a case-insensitive manner. The resulting index supports several analytical queries, including retrieving the frequency of a specific word, identifying the top-K most frequent words, and computing the difference in word frequency between two indexed versions of files.

2 System Architecture

The system is designed using an object-oriented approach consisting of four main classes: FileBufferReader, TextTokenizer, VersionedIndex, and QueryProcessor. Each class has a clearly defined responsibility in the indexing and query pipeline.

2.1 FileBufferReader

The FileBufferReader class handles incremental file reading using a fixed-size buffer. Instead of loading the entire dataset, the class reads the file in chunks whose size is determined by the buffer parameter provided through the command line.

This design ensures that memory usage remains bounded by the buffer size. Regardless of the size of the dataset, the program only stores the current buffer chunk and the indexing structure in memory.

2.2 TextTokenizer

The TextTokenizer class is responsible for extracting words from the text data contained in each buffer chunk. The tokenizer scans characters sequentially and groups together contiguous alphanumeric characters to form words.

All extracted tokens are converted to lowercase in order to maintain case-insensitive indexing. This ensures that variations such as “Error”, “ERROR”, and “error” are treated as the same word.

A key challenge arises when a word is split across two buffer chunks. To address this, the tokenizer maintains a leftover string that stores the incomplete portion of a word encountered at the end of a buffer. When the next buffer is read, this fragment is combined with the new characters to reconstruct the complete token.

2.3 VersionedIndex

The VersionedIndex class maintains the mapping between words and their frequencies for each indexed version of the file. Each version corresponds to a unique identifier provided through the command line.

Internally, the class stores a frequency map for each version. This structure allows multiple datasets to be indexed within the same program execution and enables comparison queries between versions.

2.4 QueryProcessor

The QueryProcessor class manages query execution. It interprets the command-line parameters and performs the appropriate operation on the stored index.

Three types of queries are supported: word frequency queries, top-K frequent word queries, and difference queries between two indexed versions. The query mechanism is implemented using an abstract base class that defines a virtual execution function. Specific query implementations inherit from this base class.

3 Object-Oriented Implementation

Several object-oriented programming features of C++ are used in the implementation.

Runtime polymorphism is achieved through an abstract base class that defines a virtual query execution function. Derived classes implement the logic for specific queries, such as word queries, top-K queries, and difference queries. A base-class pointer is used to call the right query implementation dynamically at runtime.

Function overloading is used in the command-line argument parsing functions. Two versions of the argument retrieval function let the program extract both string and numeric parameters from the command line.

User-defined templates are used to manage the word-frequency mapping structure. Templates keep the indexing structure generic and reusable while maintaining efficiency.

Exception handling manages runtime errors. The program throws exceptions when invalid buffer sizes are provided, when required command-line arguments are missing, or when input files cannot be opened. This ensures that the program shuts down safely while offering clear error messages.

4 Command-Line Interface

The program accepts input parameters through command-line flags that specify the input file, version identifiers, buffer size, query type, and query parameters.

Example commands include:

```
./analyzer --file dataset.txt --version v1
--buffer 512 --query word --word error

./analyzer --file dataset.txt --version v1
--buffer 512 --query top --top 10

./analyzer --file1 dataset_v1.txt --version1 v1
--file2 dataset_v2.txt --version2 v2
--buffer 512 --query diff --word error
```

These flags allow users to execute different analytical queries on indexed datasets without modifying the source code.

5 Performance Metrics

To evaluate system performance, the program reports runtime metrics after executing a query. These metrics include the buffer size allocated for processing and the total execution time measured using high-resolution timers.

Allocated buffer size: [reported during execution]

Execution time: [reported during execution]

These metrics provide insight into the efficiency of the incremental processing approach when applied to large datasets.

6 Conclusion

The system in place shows an effective way to index large text files with incremental processing. By using a fixed-size buffer and not loading the entire file, the program keeps memory use separate from the file size. The modular, object-oriented design makes maintenance easier and allows for different query types through runtime polymorphism. The use of templates, function overloading, and exception handling adds to a strong and flexible implementation.